TASK 3 – Type Conversion and Precision (4 Marks)

File: Task03/precision.c

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a = 5, b = 2;
    int units = 150;
    long large = (long)INT_MAX + 100L;

    printf("5 / 2 = %d\\n", a / b);
    printf("150 / 100 = %d\\n", units / 100);
    printf("0.1f + 0.2f = %.9f\\n", 0.1f + 0.2f);
    printf("Large long value = %ld\\n", large);
    printf("Casting large long to int is not safe when out of range.\\n");
    return 0;
}
```

Unexpected expressions and explanations

5 / 2 prints 2, not 2.5, because both operands are integers and integer division discards the fractional part.

150 / 100 prints 1, not 1.5. Convert to floating point before division if a fractional result is truly needed; billing slabs should instead use integer comparisons and integer paise.

0.1f + 0.2f may print approximately 0.300000012 because many decimal fractions cannot be represented exactly in binary floating point.

Converting a long value outside the range of int to int is implementation-defined in C. A cast does not make the value fit safely; choose a type that can hold the value.

Rounding and complexity

Use integer paise for currency: Rs 2.25 = 225 paise and Rs 4.50 = 450 paise. If a tariff ever produces fractional paise, define a rule explicitly, such as nearest paise with exact half-paise values rounded upward. Do not round inconsistently at different steps.

A float-based solution rounds the final result only after floating-point calculations. Its error may accumulate with the number of bills n; a rough bound for independent per-bill rounding of at most 0.5 paise is 0.5n paise in aggregate, while floating-point representation error depends on the values and operations. Integer paise is exact within the integer type's range. Both approaches use O(1) time and space per bill, but integer arithmetic is more reliable for money.